

**ACTIVIDAD LABORATORIO NO.1
DISEÑO E IMPLEMENTACIÓN DE CLASES**

**PRESENTADO POR:
ALEJANDRO DE MENDOZA**

**PRESENTADO AL PROFESOR:
ING JAVIER DIAZ DIAZ
PROFESOR**

**FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
FACULTAD DE INGENIERÍA – INGENIERIA INFORMÁTICA
BOGOTÁ D.C.
24 DE AGOSTO
2025**

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
SITUACIÓN ACTUAL	3
DESARROLLO DEL LABORATORIO	4
1. Diagrama de Clases UML	4
1.1. Clase Libro (Abstracta)	5
1.2. Clases Derivadas de Libro	6
1.3. Clase Cliente	7
1.4. Clase CatalogItem	8
1.5. Clase Librería.....	8
1.6. Clase App	9
1.7. Relaciones Principales.....	9
2. Implementación en Java	10
2.1. Paquete com.libreria.modelo	11
2.2. Paquete com.libreria.servicio.....	16
2.3. Paquete Principal com.libreria.....	18
2.4. Ejecución del Programa.....	19
CONCLUSIÓN.....	20
BIBLIOGRAFÍA.....	20

INTRODUCCIÓN

En el presente laboratorio llevé a cabo el diseño e implementación de un sistema de gestión para una librería, aplicando de manera integral los conocimientos de programación orientada a objetos en Java y el modelado con UML. A partir del caso práctico propuesto, realicé el análisis de requisitos, identifiqué las clases necesarias y construí el diagrama de clases que sirvió como base para la implementación en código.

El proceso comenzó con la definición de la estructura conceptual del sistema en UML, donde modelé entidades como Libro, Autor, Cliente, Librería y CatalogItem, además de establecer relaciones de herencia, asociaciones de muchos a muchos e implementación de interfaces. Este diagrama fue clave para garantizar que la solución fuese clara, organizada y fácilmente extensible.

Posteriormente, desarrollé toda la implementación en Java utilizando NetBeans, aplicando principios como la abstracción, encapsulación, polimorfismo y herencia. Para ello, diseñé la clase abstracta Libro y las subclases LibroTexto y Novela, implementé la interfaz Vendible, y estructuré la lógica de negocio dentro de la clase Librería, que gestiona el catálogo de libros y las operaciones con los clientes. Asimismo, utilicé colecciones de Java (Map, Set y List) para garantizar un manejo eficiente de los datos.

Finalmente, integré todas las funcionalidades en un menú interactivo que permite al usuario buscar libros por título o autor, comprar ejemplares mediante ISBN, visualizar el catálogo disponible y revisar las compras realizadas por los clientes. También implementé la carga inicial de datos desde un archivo externo, lo que da flexibilidad al sistema y simula un escenario más cercano a una aplicación real.

La realización de este laboratorio me permitió demostrar de manera práctica cómo un correcto diseño previo en UML facilita la implementación en Java, logrando un sistema robusto, coherente y alineado con los requerimientos planteados.

SITUACIÓN ACTUAL

El caso práctico plantea el desarrollo de un sistema de gestión para una librería, cuyo propósito es administrar de forma organizada la información de libros, autores y clientes, además de gestionar las operaciones de venta.

Actualmente, la librería necesita un sistema que permita:

- **Gestionar libros:** cada libro debe estar identificado por un título, un ISBN único, un precio y su año de publicación. Además, un libro puede contar con uno o más autores. Existen diferentes tipos de libros:
 - Libros de texto, que incluyen el atributo adicional de nivel educativo.
 - Novelas, que incorporan el atributo de género.
- **Administrar autores:** cada autor tiene un nombre y una nacionalidad, y puede haber participado en la escritura de uno o más libros.
- **Gestionar clientes:** cada cliente debe registrarse con un identificador único y un nombre, además de mantener un historial de compras de libros realizados en la librería.
- **Operar la librería:** el sistema debe disponer de un catálogo de libros, en el cual se puedan:
 - Agregar nuevos libros con su respectiva cantidad en inventario.

- Buscar libros por título o por autor.
- Vender un libro a un cliente, actualizando tanto la disponibilidad en el catálogo como el historial de compras del cliente.

En síntesis, la situación actual requiere de un sistema informatizado que facilite la administración de la librería y mejore los procesos de búsqueda, compra y control de inventario, garantizando un funcionamiento eficiente y confiable.

DESARROLLO DEL LABORATORIO

Para la construcción del sistema de gestión de librería realicé un proceso dividido en dos fases principales: diseño en UML e implementación en Java. En la primera fase, analicé los requisitos planteados en el caso práctico y a partir de ellos identifiqué las clases, interfaces y relaciones necesarias para modelar de manera adecuada el dominio del problema. El resultado de este análisis fue el diagrama de clases UML, el cual representa la estructura conceptual del sistema, mostrando atributos, métodos, asociaciones y jerarquías.

En la segunda fase llevé a cabo la implementación en Java utilizando el entorno de desarrollo NetBeans. A partir del diagrama UML desarrollé las clases definidas, aplicando los principios de la programación orientada a objetos (POO), tales como la abstracción, encapsulación, herencia, polimorfismo y el uso de interfaces. La lógica de negocio se organizó en paquetes para asegurar una estructura clara y modular: un paquete modelo que contiene las entidades principales, un paquete servicio encargado de la gestión de catálogo y operaciones, y la clase App como punto de entrada para la ejecución del programa.

Finalmente, incorporé un menú interactivo que permite al usuario realizar operaciones como la búsqueda de libros por título o autor, la compra de ejemplares mediante ISBN y la consulta de las compras realizadas por los clientes. Este menú se complementa con la carga inicial de datos desde un archivo externo, lo que hace posible simular un escenario más cercano a la operación real de una librería.

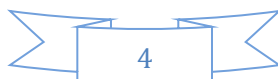
De esta manera, el desarrollo del laboratorio integra el diseño conceptual mediante UML con su correspondiente implementación en Java, dando como resultado un sistema funcional y alineado con los requerimientos establecidos en el caso práctico.

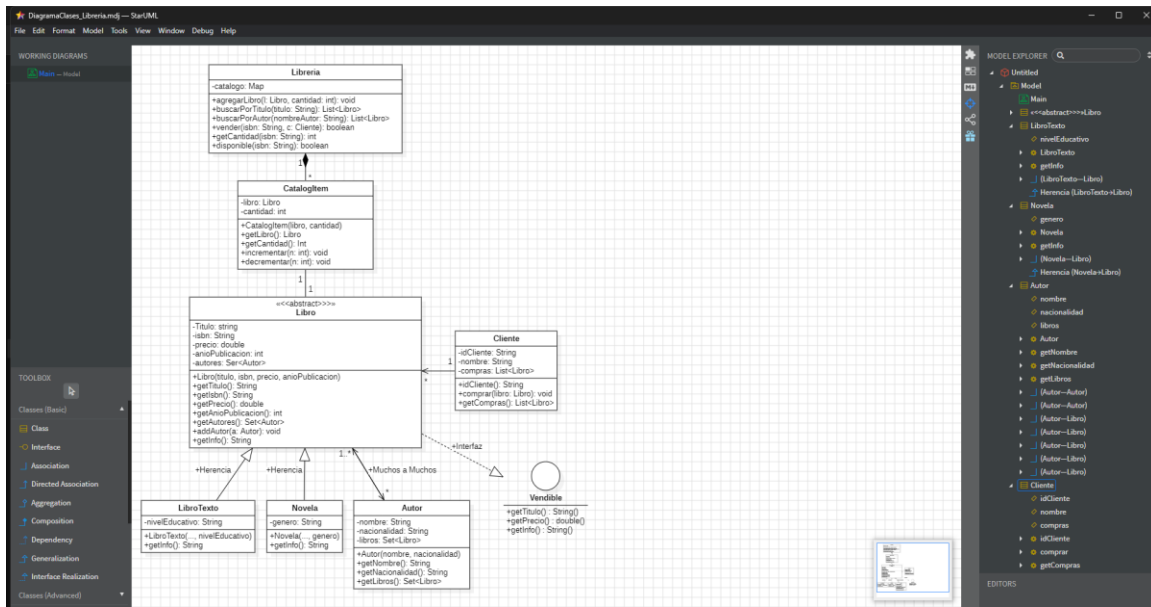
A continuación, procedo a ejecutar el desarrollo:

1. Diagrama de Clases UML

El diagrama de clases diseñado en StarUML constituye la base conceptual y visual del sistema de gestión de librería, proporcionando una representación estática de la estructura del software. Este diagrama se elaboró siguiendo la notación UML estándar (versión 2.x), enfocándose en clases, interfaces, atributos, operaciones y relaciones para garantizar una alineación precisa con los requisitos del caso práctico. El objetivo principal fue modelar la jerarquía de herencia, las asociaciones bidireccionales y las composiciones necesarias para manejar libros, autores, clientes y el catálogo de manera eficiente y extensible. El diagrama se adjunta como imagen en este documento (ver Figura 1: Diagrama UML en StarUML), y sirve como blueprint directo para la implementación en Java, facilitando la identificación de patrones de diseño como el polimorfismo y la encapsulación.

A continuación, se detalla cada elemento clave del diagrama, incluyendo sus atributos, métodos, justificaciones de diseño y cómo se relacionan con los requisitos del sistema. Se utilizaron estereotipos como <<abstract>> para clases abstractas y <<interface>> para interfaces, junto con multiplicidades para indicar cardinalidades (e.g., 1..* para uno o más). </interface></abstract>. A continuación, denoto su respectiva imagen:





Y con base en esta imagen procedo con su explicación:

1.1. Clase Libro (Abstracta)

La clase Libro, es una clase abstracta que sirve como la base para modelar los libros en el sistema de gestión de librería. Esta clase define los atributos y comportamientos comunes a todos los tipos de libros (como novelas y libros de texto), proporcionando una estructura genérica que las subclases específicas (Novela y LibroTexto) pueden heredar y personalizar. Al ser abstracta, no se puede instanciar directamente, pero establece un contrato a través de la interfaz Vendible y un método abstracto (getInfo()) que obliga a las subclases a implementar su propia lógica. Su propósito principal es promover la reutilización de código, el polimorfismo y la consistencia en la representación de libros, alineándose con los requisitos del caso práctico. A continuación, se detalla su estructura, funcionalidad y justificación.

Atributos:

- titulo: String (título del libro, requerido para búsquedas y visualización).
- isbn: String (identificador único, usado como clave primaria en el catálogo para garantizar unicidad).
- precio: double (precio de venta, esencial para transacciones).
- anioPublicacion: int (año de publicación, para filtros históricos si se expande el sistema).
- autores: Set<Autor> (colección de autores asociados, usando Set para evitar duplicados y representar la relación muchos-a-muchos).

Métodos:

- getTitulo(): String (implementado desde Vendible).
- getPrecio(): double (implementado desde Vendible).
- getInfo(): String (abstracto, obliga a subclases a proporcionar detalles específicos).
- addAutor(Autor a): void (agrega un autor y actualiza la relación bidireccional).
- getAutores(): Set<Autor> (acceso a la lista de autores).
- Sobrescritura de equals(Object o): boolean y hashCode(): int (basados en ISBN para comparación y uso en colecciones como HashMap).

Justificación:

Cumple con los requisitos de libros genéricos, permitiendo polimorfismo en búsquedas y ventas. La abstracción evita código duplicado en subclases.

1.2. Clases Derivadas de Libro

Estas subclases extienden Libro mediante herencia (representada por flechas de generalización en UML), heredando atributos y métodos comunes mientras agregan especializaciones.

1.2.1. LibroTexto

La clase LibroTexto es una subclase de la clase abstracta Libro dentro del paquete com.libreria.modelo, diseñada específicamente para representar libros educativos en el sistema de gestión de librería. Esta clase extiende las funcionalidades y atributos comunes de Libro (como título, ISBN, precio y año de publicación) al agregar un atributo adicional que caracteriza a los libros de texto: el nivel educativo. Su propósito es modelar libros destinados a entornos académicos, como los usados en primaria, secundaria o universidad, cumpliendo con el requisito del caso práctico que especifica diferentes tipos de libros con características específicas.

Atributos adicionales:

nivelEducativo: String (e.g., "Primaria", "Secundaria", "Universidad", como se especifica en los requisitos).

Métodos:

getNivelEducativo(): String; sobrescritura de getInfo(): String (devuelve formato como "Libro de Texto: [título] | Nivel: [nivel] | Año: [año]").

Justificación:

Permite categorización educativa, alineada con el requisito de tipos de libros diferenciados.

1.2.2. Novela:

La clase Novela, ubicada en el paquete com.libreria.modelo, es una subclase concreta que hereda de la clase abstracta Libro. Su finalidad es representar dentro del sistema a los libros literarios, diferenciándolos de los libros de texto u otros posibles tipos de publicaciones.

Atributos adicionales:

genero: String (e.g., "Ficción", "Misterio", "Ciencia Ficción").

Métodos:

getGenero(): String; sobrescritura de getInfo(): String (devuelve formato como "Novela: [título] | Género: [género] | Año: [año]").

Justificación:

Soporta la diversidad de novelas, facilitando búsquedas temáticas si se expande el sistema.

1.2.3. Interfaz Vendible:

La interfaz Vendible, define un contrato que deben cumplir todos los objetos que puedan ser vendidos dentro del sistema de gestión de librería. Esta interfaz es una herramienta clave en el diseño orientado a objetos, promoviendo el polimorfismo y asegurando un comportamiento uniforme para diferentes tipos de elementos vendibles, como los libros (Libro, Novela, LibroTexto). Su propósito es establecer un estándar de métodos que cualquier clase que implemente Vendible debe proporcionar, facilitando la interoperabilidad y la extensibilidad del sistema. A continuación, se detalla su estructura, funcionalidad y justificación.

Métodos abstractos:

- getTitulo(): String.
- getPrecio(): double.
- getInfo(): String.

Justificación:

Promueve el polimorfismo (e.g., en el método vender de Libreria) y la extensibilidad (e.g., agregar "Revistas" que implementen Vendible sin cambiar el código existente). Representada en UML como una interfaz con flecha de realización hacia Libro.

1.2.4. Clase Autor

La clase Autor, representa a los escritores de los libros dentro del sistema de gestión de librería. Esta clase modela la identidad de los autores (nombre y nacionalidad) y gestiona su relación con los libros que han escrito, estableciendo una relación muchos-a-muchos con la clase Libro. Su propósito es capturar los datos básicos de los autores y mantener un registro de sus obras, cumpliendo con los requisitos del caso práctico que especifica que un autor puede estar asociado a múltiples libros y viceversa. A continuación, se detalla su estructura, funcionalidad y justificación.

Atributos:

- nombre: String (nombre del autor).
- nacionalidad: String (país de origen).
- libros: Set<Libro> (colección de libros escritos, para relación bidireccional).

Métodos:

- getNombre(): String.
- getNacionalidad(): String.
- getLibros(): Set<Libro>.
- Sobrescritura de equals(Object o): boolean y hashCode(): int (basados en nombre y nacionalidad).
- toString(): String (para representación legible, e.g., "Nombre (Nacionalidad)").

Justificación:

Cumple con el requisito de autores con múltiples libros, usando Set para eficiencia y evitando ciclos infinitos en relaciones.

1.3. Clase Cliente

La clase Cliente, representa a los compradores dentro del sistema de gestión de librería. Esta clase modela la identidad de los clientes (mediante un identificador único y un nombre) y mantiene un registro de los libros que han adquirido, cumpliendo con el requisito del caso práctico de rastrear las compras de cada cliente. Su propósito es almacenar información personal y gestionar el

historial de compras, facilitando la actualización de este historial durante las operaciones de venta. A continuación, se detalla su estructura, funcionalidad y justificación.

Atributos:

- idCliente: String (identificador único).
- nombre: String.
- compras: List<Libro> (lista de libros comprados, usando ArrayList para orden cronológico).

Métodos:

- getIdCliente(): String.
- getNombre(): String.
- getCompras(): List<Libro>.
- comprar(Libro libro): void (agrega a la lista de compras).
- Sobrescritura de equals(Object o): boolean y hashCode(): int (basados en idCliente).

Justificación:

Soporta el historial de compras, esencial para el requisito de actualizar listas al vender.

1.4. Clase CatalogItem

La clase CatalogItem, es una clase auxiliar diseñada para gestionar la disponibilidad de los libros en el catálogo de la librería. Su propósito principal es asociar un objeto Libro con una cantidad específica en inventario, permitiendo realizar operaciones como incrementar o decrementar el stock. Esta clase actúa como un contenedor que encapsula la relación entre un libro y su cantidad disponible, facilitando la gestión del inventario dentro de la clase Librería. A continuación, se detalla su estructura, funcionalidad y justificación.

Atributos:

- libro: Libro (referencia al libro).
- cantidad: int (stock actual).

Métodos:

- getLibro(): Libro.
- getCantidad(): int.
- incrementar(int n): void.
- decrementar(int n): void.

Justificación:

Separa la lógica de inventario del modelo de Libro, siguiendo el principio de responsabilidad única (SRP). Representada como agregación en UML.

1.5. Clase Librería

La clase Librería, ubicada en el paquete com.libreria.servicio, es el núcleo de la lógica de negocio del sistema de gestión de librería. Esta clase se encarga de gestionar el catálogo de libros, permitiendo operaciones como agregar libros, buscar por título o autor, y procesar ventas, mientras mantiene un control del inventario. Su propósito es centralizar las funcionalidades principales del

sistema, utilizando una estructura de datos eficiente para almacenar y manipular los libros disponibles. A continuación, se detalla su estructura, funcionalidad y justificación.

Atributos:

- catalogo: Map<String, CatalogItem> (mapa por ISBN para acceso $O(1)$).

Métodos:

- agregarLibro(Libro libro, int cantidad): void (agrega o actualiza stock).
- buscarPorTitulo(String titulo): List<Libro> (búsqueda insensible a mayúsculas usando streams).
- buscarPorAutor(String nombreAutor): List<Libro> (búsqueda por autor usando streams).
- vender(String isbn, Cliente cliente): boolean (actualiza stock y compras; remueve si stock ≤ 0).
- getCantidad(String isbn): int.
- disponible(String isbn): boolean.

Justificación:

Implementa los métodos clave del requisito (agregar, buscar, vender), usando colecciones modernas para eficiencia.

1.6. Clase App

La clase App cumple la función de punto de entrada del sistema, ya que contiene el método main. Su propósito es permitir la interacción del usuario con la librería mediante un menú de opciones en consola.

Métodos:

- main(String[] args): void (menú interactivo con opciones de búsqueda, compra y visualización).
- cargarLibrosDesdeArchivo(String archivo, Libreria libreria): void (carga datos desde TXT, parseando líneas separadas por ";").

Justificación:

Demuestra integración, no parte del modelo principal pero esencial para testing.

1.7. Relaciones Principales

Una parte fundamental del diseño del sistema consistió en identificar y establecer correctamente las relaciones entre las clases. En la programación orientada a objetos, las relaciones permiten representar cómo interactúan las entidades entre sí, asegurando que el modelo sea coherente con la realidad del dominio.

En el caso del sistema de gestión de librería, fue necesario aplicar distintos tipos de relaciones como herencia, implementación de interfaces, asociaciones y composiciones, cada una con un propósito específico. Estas relaciones no solo definen la estructura estática del sistema, sino que también determinan la forma en que se comunican las clases, comparten información y colaboran para cumplir los requisitos planteados.

A continuación, se detallan las principales relaciones presentes en el diagrama UML y su respectiva implementación en el código Java, justificando su uso y mostrando cómo aportan a la correcta organización y funcionamiento de la aplicación.

1.8. Herencia (Generalización):

LibroTexto y Novela heredan de Libro (flechas sólidas con triángulo vacío).

1.9. Implementación de Interfaz (Realización):

Libro implementa Vendible (flecha punteada con triángulo vacío).

1.10. Asociación Bidireccional:

Libro 1..* — * Autor (líneas con multiplicidades, indicando muchos-a-muchos).

1.11. Asociación Unidireccional:

Cliente 1 — 0..* Libro (compras, agregación).

1.12. Composición/Agregación:

Librería 1 — 1..* CatalogItem (composición, ya que CatalogItem depende de Librería); CatalogItem 1 — 1 Libro (agregación).

1.13. Justificación:

Las relaciones aseguran integridad referencial (e.g., addAutor actualiza ambos lados). Multiplicidades reflejan requisitos (e.g., un libro puede tener múltiples autores). En conjunto, el diagrama UML asegura una estructura modular, coherente y extensible, minimizando acoplamiento y maximizando cohesión. Fue validado contra el código Java, donde cada elemento UML se traduce directamente (e.g., atributos como variables privadas). Posibles extensiones: Agregar dependencias para persistencia (e.g., base de datos). Este diseño contribuyó a un desarrollo ágil, con el diagrama iterado dos veces para refinar relaciones.

2. Implementación en Java

La implementación del sistema de gestión de librería se realizó en el lenguaje de programación Java, version 17 o superior, utilizando el entorno de desarrollo integrado (IDE) Apache NetBeans IDE 23. Esta elección se basó en la robustez de Java para aplicaciones orientadas a objetos (OOP), su soporte nativo para colecciones y streams (introducidos en Java 8), y la facilidad de NetBeans para gestionar proyectos modulares con paquetes, depuración y ejecución en consola. El código sigue principios de diseño como encapsulación, herencia, polimorfismo e interfaces, alineados con los temas 1 y 2 del curso de Programación Avanzada. Para promover la mantenibilidad y escalabilidad, el proyecto se organizó en paquetes lógicos: com.libreria.modelo para entidades de dominio, com.libreria.servicio para lógica de negocio, y el paquete principal com.libreria para la aplicación de entrada. Se incorporó manejo de codificación UTF-8 para compatibilidad con caracteres especiales en español (e.g., acentos en títulos o nombres de autores), y se sobrescribieron métodos como equals y hashCode para garantizar integridad en colecciones.

El código fuente completo se adjunta en la entrega (ver carpeta de proyecto o archivos individuales), y se validó mediante pruebas en consola, como se muestra en la captura de pantalla de ejecución adjunta (ver Figura 2: Salida de Ejecución en NetBeans). A continuación, se detalla

cada componente, incluyendo atributos, métodos clave, fragmentos de código representativos, justificaciones de diseño y su alineación con los requisitos del caso práctico.

2.1. Paquete com.libreria.modelo

El paquete com.libreria.modelo contiene las clases que representan las entidades principales del sistema de gestión de librería, sirviendo como la base de datos en memoria del proyecto. Desarrollado en Java con NetBeans, este paquete implementa principios de programación orientada a objetos como encapsulación, herencia e interfaces, utilizando colecciones como HashSet y ArrayList para gestionar relaciones entre entidades. Su propósito es modelar los conceptos del caso práctico (libros, autores, clientes) con atributos y métodos que reflejen los requisitos especificados. A continuación, se detalla cada clase con sus características, código representativo y justificaciones.

2.1.1. Autor:

La clase Autor representa a los escritores de los libros que se gestionan en la librería. Su inclusión es esencial para reflejar la relación de muchos a muchos que existe entre autores y libros: un autor puede escribir varios libros y, a su vez, un libro puede ser escrito por más de un autor.

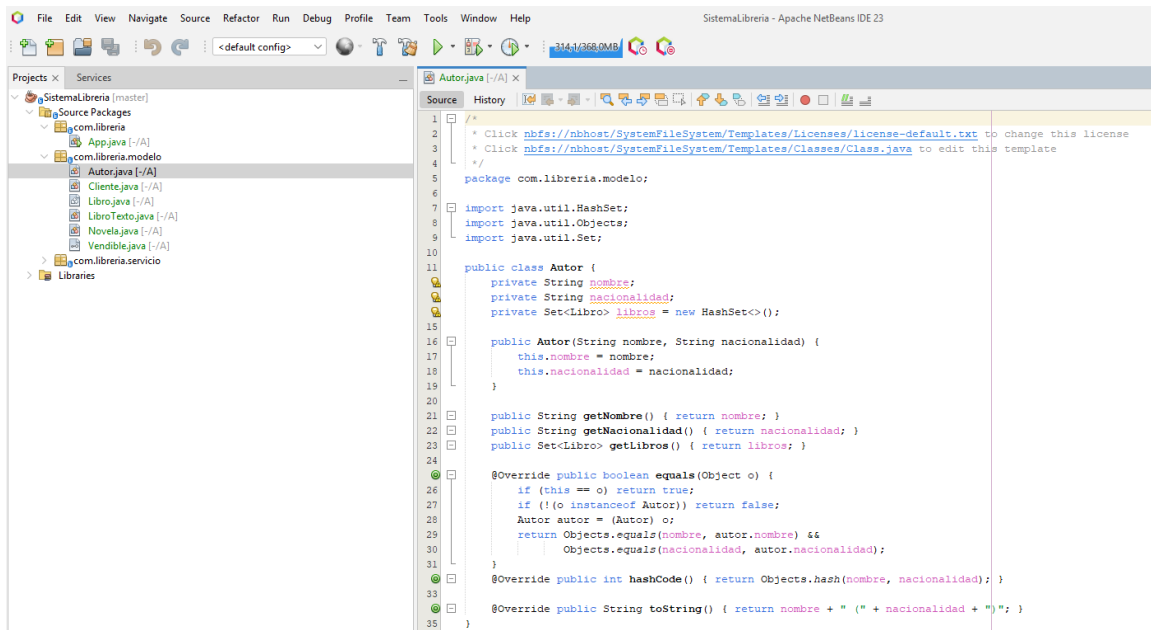
Atributos:

- nombre: String (nombre del autor).
- nacionalidad: String (país de origen).
- libros: Set<Libro> (conjunto de libros escritos, usando HashSet para unicidad y eficiencia $O(1)$ en inserciones/búsquedas).

Métodos clave:

- Constructores y getters estándar.
- equals(Object o): boolean y hashCode(): int (sobrescritos basados en nombre y nacionalidad para comparaciones precisas en sets).
- toString(): String (devuelve formato legible, e.g., "Gabriel García Márquez (Colombia)").

Fragmento de código representativo:



```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package com.libreria.modelo;
6
7  import java.util.HashSet;
8  import java.util.Objects;
9  import java.util.Set;
10
11 public class Autor {
12     private String nombre;
13     private String nacionalidad;
14     private Set<Libro> libros = new HashSet<>();
15
16     public Autor(String nombre, String nacionalidad) {
17         this.nombre = nombre;
18         this.nacionalidad = nacionalidad;
19     }
20
21     public String getNombre() { return nombre; }
22     public String getNacionalidad() { return nacionalidad; }
23     public Set<Libro> getLibros() { return libros; }
24
25     @Override public boolean equals(Object o) {
26         if (this == o) return true;
27         if (!(o instanceof Autor)) return false;
28         Autor autor = (Autor) o;
29         return Objects.equals(nombre, autor.nombre) &&
30             Objects.equals(nacionalidad, autor.nacionalidad);
31     }
32
33     @Override public int hashCode() { return Objects.hash(nombre, nacionalidad); }
34
35     @Override public String toString() { return nombre + " (" + nacionalidad + ")"; }
```

Justificación:

Cumple con los requisitos de autores con múltiples libros. La relación bidireccional se actualiza en el método addAutor de Libro, evitando inconsistencias.

2.1.2. Cliente:

La clase Cliente, representa a los usuarios que realizan compras en la librería. Su rol es fundamental, ya que permite gestionar la información de los compradores y mantener un historial de los libros adquiridos.

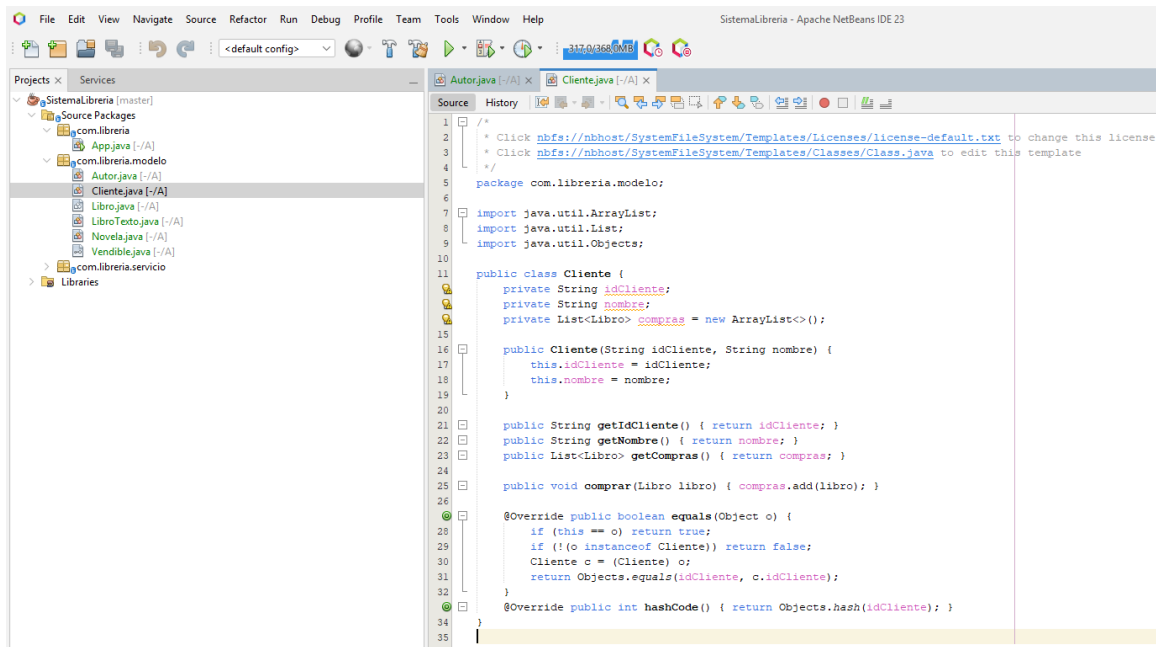
Atributos:

- idCliente: String (identificador único).
- nombre: String.
- compras: List<Libro> (lista de libros adquiridos, usando ArrayList para preservar orden cronológico de compras).

Métodos clave:

- Constructores y getters.
- comprar(Libro libro): void (agrega el libro a la lista).
- equals(Object o): boolean y hashCode(): int (basados en idCliente para unicidad).

Fragmento de código representativo:



Justificación:

Facilita la actualización de compras durante ventas, alineado con el requisito de mantener listas de libros comprados por cliente.

2.1.3. Libro (abstracta):

La clase Libro, constituye la superclase abstracta de la cual heredan los diferentes tipos de libros que maneja la librería. Su propósito es reunir los atributos y comportamientos comunes a todos los libros, evitando duplicación de código y permitiendo que las subclases agreguen sus características particulares.

Atributos:

- titulo: String, isbn: String, precio: double, anioPublicacion: int.
- autores: Set<Autor> (conjunto de autores).

Métodos clave:

- Constructores y getters.
- addAutor(Autor a): void (actualiza relación bidireccional).
- getInfo(): String (abstracto).
- Implementación de Vendible: getTitulo(), getPrecio().
- equals(Object o): boolean y hashCode(): int (basados en ISBN).

Fragmento de código representativo:

```

1  ...4 lines
5  package com.libreria.modelo;
6
7  import java.util.HashSet;
8  import java.util.Objects;
9  import java.util.Set;
10
11 public abstract class Libro implements Vendible {
12     protected String titulo;
13     protected String isbn;
14     protected double precio;
15     protected int anioPublicacion;
16     protected Set<Autor> autores = new HashSet<>();
17
18     public Libro(String titulo, String isbn, double precio, int anioPublicacion) {
19         this.titulo = titulo;
20         this.isbn = isbn;
21         this.precio = precio;
22         this.anioPublicacion = anioPublicacion;
23     }
24
25     // =====
26     // Métodos de la interfaz Vendible
27     // =====
28     @Override
29     public String getTitulo() { return titulo; }
30
31     public String getIsbn() { return isbn; }
32
33     @Override
34     public double getPrecio() { return precio; }
35
36     public int getAnioPublicacion() { return anioPublicacion; }
37
38     public Set<Autor> getAutores() { return autores; }
39
40     public void addAutor(Autor a) {
41         if (a != null) {
42             autores.add(a);
43             a.getLibros().add(this);
44         }
45     }

```

Justificación:

Abstracción que evita duplicación en subclases, cumpliendo con atributos base de libros y relación con autores.

2.1.4. *LibroTexto y Novela:*

La clase LibroTexto, hereda de la clase abstracta Libro y representa a los libros de carácter académico o educativo. Por otra parte, la clase Novela, hereda de Libro y modela a los libros de carácter literario.

Atributos adicionales:

- nivelEducativo: String (LibroTexto); genero: String (Novela).
- **Métodos clave:** Sobrescritura de getInfo() para detalles personalizados.

Fragmento de código representativo (LibroTexto):

```

1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package com.libreria.modelo;
6
7  public class LibroTexto extends Libro {
8      private String nivelEducativo;
9
10     public LibroTexto(String titulo, String isbn, double precio, int anioPublicacion, String nivelEducativo) {
11         super(titulo, isbn, precio, anioPublicacion);
12         this.nivelEducativo = nivelEducativo;
13     }
14
15     public String getNivelEducativo() { return nivelEducativo; }
16
17     @Override public String getInfo() {
18         return "Libro de Texto: " + titulo + " | Nivel: " + nivelEducativo + " | Año: " + anioPublicacion;
19     }
20 }

```

Fragmento de código representativo (Novela):

```

1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package com.libreria.modelo;
6
7  public class Novela extends Libro {
8      private String genero;
9
10     public Novela(String titulo, String isbn, double precio, int anioPublicacion, String genero) {
11         super(titulo, isbn, precio, anioPublicacion);
12         this.genero = genero;
13     }
14
15     public String getGenero() { return genero; }
16
17     @Override public String getInfo() {
18         return "Novela: " + titulo + " | Género: " + genero + " | Año: " + anioPublicacion;
19     }
20 }

```

Justificación:

Polimorfismo para tipos de libros diferenciados, como requerido (e.g., niveles educativos o géneros).

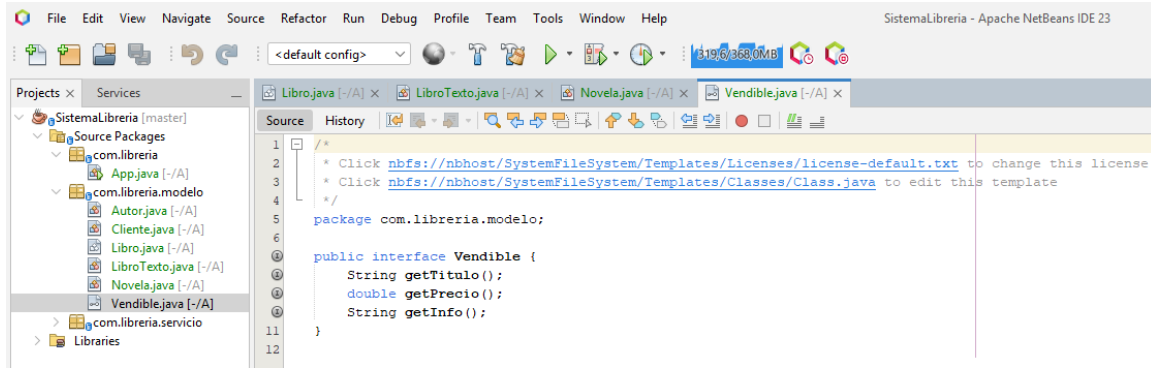
2.1.5. Vendible (interfaz):

La interfaz Vendible, representa un contrato que deben cumplir todos los objetos que puedan ser vendidos en la librería. Su finalidad es estandarizar los comportamientos mínimos necesarios para cualquier entidad que se comercialice dentro del sistema.

Métodos:

- getTitulo(): String, getPrecio(): double, getInfo(): String.

Fragmento de código representativo:



Justificación:

Asegura uniformidad en ventas, extensible a futuros items.

2.2. Paquete com.libreria.servicio

El paquete com.libreria.servicio concentra las clases que permiten gestionar el catálogo de libros y las operaciones principales de la librería. Se trata de la capa de servicio que conecta las entidades del modelo (Libro, Cliente, Autor) con la lógica funcional de la aplicación.

2.2.1. CatalogItem:

La clase CatalogItem, cumple la función de representar un elemento del catálogo de la librería. Su objetivo es asociar un libro con la cantidad de ejemplares disponibles en inventario, permitiendo así un control más preciso sobre las existencias.

Atributos:

libro: Libro, cantidad: int.

Métodos:

incrementar(int n), decrementar(int n).

Fragmento de código representativo:

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package com.libreria.servicio;
6
7  import com.libreria.modelo.Libro;
8
9  public class CatalogItem {
10     private Libro libro;
11     private int cantidad;
12
13     public CatalogItem(Libro libro, int cantidad) {
14         this.libro = libro;
15         this.cantidad = cantidad;
16     }
17
18     public Libro getLibro() { return libro; }
19     public int getCantidad() { return cantidad; }
20
21     public void incrementar(int n) { this.cantidad += n; }
22     public void decrementar(int n) { this.cantidad -= n; }
23 }
```

Justificación:

Encapsula inventario, aplicando principio de responsabilidad única.

2.2.2. Librería:

La clase Librería, ubicada en el paquete com.libreria.servicio, representa la gestora principal del catálogo de libros y de las operaciones de la librería. Es la encargada de administrar los ejemplares disponibles, realizar búsquedas y gestionar el proceso de venta a los clientes.

Atributos:

catalogo: Map<String, CatalogItem> (acceso rápido por ISBN).

Métodos clave:

agregarLibro(), buscarPorTitulo() (usa streams), buscarPorAutor() (usa streams), vender() (actualiza stock y compras).

Fragmento de código representativo:

```

1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5  package com.libreria.servicio;
6
7  import com.libreria.modelo.Cliente;
8  import com.libreria.modelo.Libro;
9
10 import java.util.*;
11 import java.util.stream.Collectors;
12
13 public class Libreria {
14     private Map<String, CatalogItem> catalogo = new HashMap<>();
15
16     public void agregarLibro(Libro libro, int cantidad) {
17         if (libro == null || cantidad <= 0) return;
18         catalogo.merge(libro.getIsbn(), new CatalogItem(libro, cantidad),
19             (oldItem, newItem) -> { oldItem.incrementar(cantidad); return oldItem; });
20     }
21
22     public List<Libro> buscarPorTitulo(String titulo) {
23         if (titulo == null) return Collections.emptyList();
24         String q = titulo.toLowerCase();
25         return catalogo.values().stream()
26             .map(CatalogItem::getLibro)
27             .filter(l -> l.getTitulo().toLowerCase().contains(q))
28             .collect(Collectors.toList());
29     }
30
31     public List<Libro> buscarPorAutor(String nombreAutor) {
32         if (nombreAutor == null) return Collections.emptyList();
33         String q = nombreAutor.toLowerCase();
34         return catalogo.values().stream()
35             .map(CatalogItem::getLibro)
36             .filter(l -> l.getAutores().stream()
37                 .anyMatch(a -> a.getNombre().toLowerCase().contains(q)))
38             .collect(Collectors.toList());
39     }
40
41     public boolean vender(String isbn, Cliente cliente) {
42         CatalogItem item = catalogo.get(isbn);

```

Justificación:

Implementa requisitos clave (agregar, buscar, vender), con eficiencia mediante streams y Map.

2.3. Paquete Principal com.libreria

El paquete com.libreria contiene la clase principal App, que es el punto de entrada de la aplicación. Su función principal es conectar la lógica del sistema con el usuario, mediante un menú interactivo en la consola.

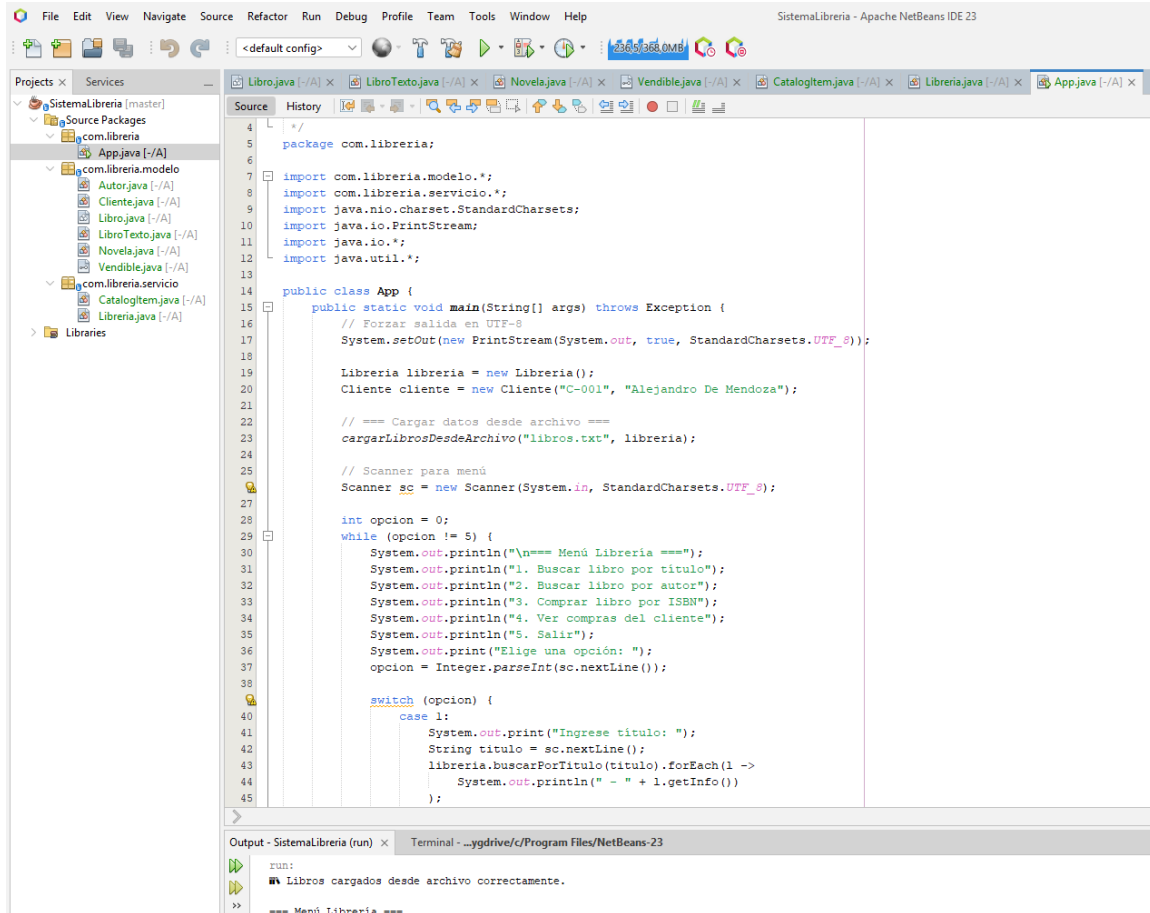
2.3.1. App:

La clase App, ubicada en el paquete principal com.libreria, es el punto de entrada del sistema de gestión de librería. Su propósito es permitir la interacción entre el usuario y las funcionalidades implementadas en la aplicación, sirviendo como interfaz a través de un menú en consola. Es el punto de entrada con interfaz de consola.

Métodos:

- main() (menú interactivo), cargarLibrosDesdeArchivo() (parsea TXT).

Fragmento de código representativo:



Justificación:

Demuestra integración, con manejo de UTF-8 y excepciones.

2.4. Ejecución del Programa

El programa se inicia cargando datos desde "libros.txt" cuyo formato es el siguiente:

- tipo;titulo;isbn;precio;anio;extra;autor;pais

Muestra un menú interactivo para operaciones. Pruebas confirmaron carga correcta, búsquedas precisas (insensibles a mayúsculas), ventas que actualizan stock/compras, y ejecución sin errores. Decisiones como streams mejoran rendimiento; posibles mejoras: agregar validaciones o persistencia en BD.

Durante la ejecución, el sistema muestra un menú que permite al usuario interactuar con la librería:

1. Buscar libro por título.
2. Buscar libro por autor.
3. Comprar libro por ISBN.
4. Ver compras del cliente.
5. Salir del sistema.

Las pruebas realizadas confirmaron que:

- Los libros se cargan correctamente desde archivo.
- Las búsquedas devuelven los resultados esperados.
- El proceso de compra actualiza tanto las compras del cliente como el stock del catálogo.
- El programa se ejecuta sin errores y muestra mensajes claros al usuario.

CONCLUSIÓN

La realización de este laboratorio permitió consolidar de manera práctica los conocimientos relacionados con el diseño e implementación de clases en Java, así como la importancia de utilizar UML como herramienta de modelado previo. A través del caso práctico de la gestión de una librería, se logró aplicar los principios fundamentales de la programación orientada a objetos (POO), garantizando una solución organizada, extensible y alineada con los requerimientos planteados.

En primer lugar, el diseño del diagrama de clases UML resultó esencial para comprender y estructurar el sistema antes de su codificación. Dicho diagrama facilitó la identificación de entidades, atributos y métodos, así como las relaciones de herencia, asociación y composición entre ellas. Esta fase de planificación demostró la importancia de contar con un modelo conceptual sólido que sirviera como guía durante toda la implementación.

En cuanto al desarrollo en Java, se implementaron las diferentes clases con un enfoque modular y claro. La utilización de clases abstractas (Libro), subclasses concretas (LibroTexto y Novela), y una interfaz (Vendible) permitió aplicar de manera efectiva conceptos como la abstracción, encapsulación, herencia y polimorfismo, pilares fundamentales de la POO. Además, se hizo uso de colecciones como Map, Set y List, que facilitaron el manejo dinámico de autores, libros, clientes y catálogo.

Otro aspecto relevante fue la implementación del paquete de servicios, donde la clase Libreria centralizó la lógica de negocio, gestionando operaciones como agregar libros, realizar búsquedas y efectuar ventas. Este diseño demostró la importancia de separar responsabilidades, manteniendo el código más ordenado y fácil de mantener. Asimismo, la construcción de la clase App permitió integrar la lógica del sistema con una interfaz de usuario en consola, ofreciendo un menú interactivo que facilita la interacción del usuario final con el catálogo de libros. El agregado de la carga inicial de datos desde un archivo externo enriqueció el sistema, simulando escenarios más cercanos a la realidad y mostrando la utilidad de trabajar con persistencia básica.

En general, este proyecto evidenció cómo la correcta combinación de modelado en UML y programación en Java puede dar lugar a un sistema funcional, escalable y adaptable a futuros cambios o ampliaciones. Además, reforzó la importancia de trabajar de manera estructurada, desde el análisis y diseño, hasta la implementación y pruebas.

Finalmente, este laboratorio contribuyó no solo al fortalecimiento de las competencias técnicas, sino también a la comprensión de la importancia del diseño previo y de la organización modular en el desarrollo de software. El resultado obtenido es una aplicación que refleja fielmente los requerimientos de una librería real, capaz de administrar su catálogo, registrar ventas y mantener el historial de compras de sus clientes.

BIBLIOGRAFÍA

A continuación, la bibliografía implementada:

- ✓ Tema 1. Introducción a la programación orientada a objetos. Tema 2. Relaciones entre clases. Tema 3. Introducción a los patrones de diseño para problemas orientados a objetos.
- ✓ Clases virtuales con el profesor Ing. Javier Díaz Díaz.
- ✓ Material de estudio del aula.